MODUL PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBYEK

MODUL 13
Penyimpanan dan Pembacaan Data ke File



Disusun oleh:

Nirwana Haidar Hari, S.Pd., M.Kom.

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
PROGRAM STUDI DI LUAR KAMPUS UTAMA SUMENEP
PENS PSDKU SUMENEP

2026

MODUL PRAKTIKUM P13
Integrasi Aplikasi Berbasis Menu dan Pengujian Fungsional

Mata Kuliah: Praktikum Pemrograman Berorientasi Obyek
Model Pembelajaran: Project Based Learning
Prasyarat: Collection, exception handling, dan penyimpanan file pada P10–P12.

1. Tujuan Praktikum

Mahasiswa mampu:

- mengintegrasikan fitur menjadi aplikasi konsol interaktif;

• memisahkan interaksi pengguna dari aturan domain;
• menjaga konsistensi data selama penggunaan aplikasi;
• menguji alur aplikasi dari input hingga penyimpanan.

2. Dasar Teori Singkat

Aplikasi berbasis menu memungkinkan pengguna menjalankan beberapa operasi dalam satu eksekusi program.
Pembagian tanggung jawab tetap dipertahankan:

Bagian	**Tanggung jawab**
Main	Menampilkan menu, membaca input, dan menampilkan hasil.
Buku	Menjaga validitas data satu buku.
Perpustakaan	Mengelola koleksi dan keunikan kode buku.
Method penyimpanan P12	Menyimpan dan memuat data dari file.

Input konsol dibaca menggunakan Scanner. Pada contoh ini, seluruh input menggunakan nextLine() agar pembacaan menu dan teks konsisten. docs.oracle.com
Prinsip utama: menu memanggil layanan objek. Aturan seperti kode unik tetap diperiksa oleh class pengelola.

3. Studi Kasus

Integrasikan program perpustakaan menjadi aplikasi dengan menu:

Pilihan	**Fitur**
1	Tambah buku
2	Tampilkan buku
3	Cari buku
4	Hapus buku
5	Simpan data
6	Muat ulang data
0	Keluar

Gunakan variabel berubah untuk menandai perubahan yang belum disimpan. • Penambahan atau penghapusan berhasil → true.
• Penyimpanan atau pemuatan berhasil → false.

• Operasi gagal → status sebelumnya dipertahankan.

4. Langkah Praktikum

Langkah 1 — Siapkan program P12

Pertahankan class Buku dan Perpustakaan, termasuk method simpan() dan muat(). Tambahkan import:
import java.util.Scanner;
Import dari P11–P12 tetap digunakan.

Langkah 2 — Ganti class Main

```
public class Main {
 private static String baca(Scanner input, String pesan) {
System.out.print(pesan);
 return input.nextLine().trim();
 }

 private static boolean konfirmasi(Scanner input) {
return baca(
 input,
 "Perubahan belum disimpan. Abaikan? Ketik YA: "
).equalsIgnoreCase("YA");
 }

 public static void main(String[] args) {
 Perpustakaan perpustakaan = new Perpustakaan();  Path
lokasi = Paths.get("buku.txt");
 boolean berubah = false;

 // Muat data lama sebelum pengguna mulai bekerja.  if
(Files.exists(lokasi)) {
 try {
 perpustakaan.muat(lokasi);
 } catch (IOException e) {
 System.out.println(
 "Data awal gagal dimuat: " + e.getMessage()  );
 return;
 }
 }

 try (Scanner input = new Scanner(System.in)) {
while (true) {
 System.out.println("\n=== PERPUSTAKAAN ===");
System.out.println("1. Tambah 2. Tampilkan");
System.out.println("3. Cari 4. Hapus");  System.out.println("5.
Simpan 6. Muat ulang");  System.out.println("0. Keluar");

 String pilihan = baca(input, "Pilihan: ");
```

```
try {
switch (pilihan) {
case "1": {
String kode = baca(input, "Kode: "); String judul = baca(input,
                            "Judul: ");

perpustakaan.tambahBuku(  new Buku(kode, judul)
);
                            berubah = true;
                           System.out.println("Buku ditambahkan.");
                            break;
}

case "2":
          perpustakaan.tampilkanBuku(); break;

case "3": {
Buku hasil = perpustakaan.cariBuku(  baca(input, "Kode yang
dicari: ")  );

System.out.println(
hasil == null
? "Buku tidak ditemukan." : "Judul: " + hasil.getJudul()
);
                            break;
}

case "4": {
String kode = baca(
input, "Kode yang dihapus: "  );

if (perpustakaan.hapusBuku(kode)) {  berubah = true;
                                System.out.println("Buku dihapus.");
} else {
System.out.println(  "Buku tidak ditemukan."  );
}
                            break;
}

case "5":
     perpustakaan.simpan(lokasi); berubah = false;
                           System.out.println("Data disimpan.");
                            break;

case "6":
if (berubah && !konfirmasi(input)) {  break;
}
```

```
                                perpustakaan.muat(lokasi);
    berubah = false;
                                System.out.println("Data dimuat ulang.");
                                break;

    case "0":
    if (berubah && !konfirmasi(input)) {  break;
    }
                                System.out.println("Program selesai.");
                                return;

    default:
    System.out.println(
    "Pilihan tidak tersedia."  );
    }

    } catch (IllegalArgumentException e) {
    System.out.println(
    "Data ditolak: " + e.getMessage()  );

    } catch (IOException e) {
    System.out.println(
    "Operasi file gagal: " + e.getMessage()  );
    }
    }
    }
    }
   }
```

Catatan desain:

• Saat file tersedia, data dimuat pada awal program.

• Jika data awal rusak, program berhenti agar pengguna dapat memperbaiki file. • Exception pada suatu menu ditangani sehingga pengguna dapat mencoba operasi lain. • Muat ulang mengganti collection dengan isi file.

• Konfirmasi diperlukan sebelum keluar atau memuat ulang ketika ada perubahan yang belum disimpan.

• Pesan berhasil ditampilkan setelah operasi selesai.

Langkah 3 — Jalankan aplikasi

javac Main.java

java Main

Gunakan direktori kerja yang sama seperti P12 agar lokasi buku.txt konsisten. **5. Pengujian Fungsional**

Lakukan pengujian berikut secara berurutan.

No.	Skenario	Hasil yang diharapkan

1	Masukkan pilihan abc	Pesan pilihan tidak tersedia; menu muncul kembali.
2	Tambahkan buku berkode baru	Buku tersimpan dalam collection.
3	Tambahkan kode yang sama	Ditolak; data sebelumnya tetap ada.
4	Cari kode yang terdaftar	Judul buku ditampilkan.
5	Hapus kode yang tidak terdaftar	Pesan tidak ditemukan; collection tetap.
6	Keluar sebelum menyimpan, lalu jawab TIDAK	Kembali ke menu.
7	Simpan, keluar, lalu jalankan kembali	Data tersimpan dimuat kembali.
8	Hapus buku, simpan, lalu jalankan kembali	Buku yang dihapus tidak muncul.
9	Tambah buku, pilih muat ulang, lalu jawab TIDAK	Perubahan tetap tersedia dalam collection.

Tambahan: rusakkan format file sebelum memilih muat ulang. Pastikan aplikasi menampilkan pesan kesalahan dan mempertahankan collection sebelumnya.

6. Tugas Proyek Kelompok

Integrasikan proyek masing-masing dengan ketentuan:

1. Sediakan menu tambah, tampilkan, cari, hapus, simpan, muat, dan keluar.
2. Tempatkan validasi domain di class yang bertanggung jawab.
3. Berikan pesan kesalahan yang menjelaskan penyebab kegagalan.
4. Tambahkan perlindungan terhadap perubahan yang belum disimpan.
5. Pisahkan kode menu ke method seperti menuTambah() dan menuCari() agar main() tetap ringkas.
6. Uji minimal **8 skenario**, mencakup operasi berhasil, gagal, serta penggunaan ulang setelah program ditutup.

Jika proyek telah menggunakan class repository, arahkan operasi file melalui repository tersebut.

Perbaikan wajib: pindahkan pemeriksaan karakter pemisah | ke saat data diterima atau dibentuk menjadi objek, sehingga data yang tidak dapat disimpan ditolak sejak awal.

7. Luaran dan Penilaian

Kumpulkan **source code, UML terbaru, contoh file data, tabel pengujian, dan panduan penggunaan singkat**.

Aspek	**Bobot**

Integrasi dan kelengkapan fitur	30%
Pembagian tanggung jawab antarclass	25%
Konsistensi data dan penanganan kesalahan	20%
Pengujian alur aplikasi	15%
Dokumentasi dan demonstrasi	10%
Total	**100%**

8. Pertanyaan Evaluasi

1. Mengapa aturan kode unik tetap ditempatkan di Perpustakaan?
2. Mengapa status berubah direset setelah penyimpanan berhasil?
3. Apa akibatnya jika status tersebut direset ketika penyimpanan gagal?
4. Mengapa pengujian setiap fitur perlu dilengkapi pengujian alur penggunaan secara utuh?

Tugas minggu depan: perbaiki temuan pengujian dan siapkan demonstrasi proyek yang mencakup alur normal, penolakan data tidak valid, serta penyimpanan dan pemuatan kembali.